Singly Linked List –
Get N^{TH} NODE 2
and Time Complexity.

[*Function to get Nth node in a Linked List*, 0 *based indexing*]

The GetNth2() function retrieves the data value stored in the node at a specified index from a linked list. To implement this, you typically traverse the linked list until you reach the node at the given index, then return its data. If the index is out of bounds, you should handle it accordingly (e.g., return null or an error message).

Program :

```
int getNthNode2(Node **head, int node)
{
    if (*head == nullptr)
    {
        cout << "Error: List is empty." << endl;
        return -1;
    }
    if (node < 0)
    {
        return -1;
    }
    else
    {
        Node *current = *head;
        int i = 0;
        while (i < node && current != nullptr)
        {
            current = current->next;
            i++;
        }
        if(current == nullptr){
            return -1;
        }
```

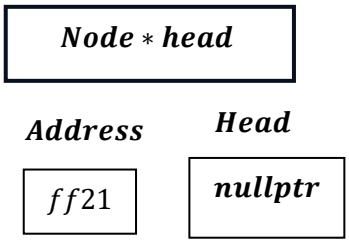
```
        else{
            return current->data;
        }
    }
}

…..
case 19:
        {
            cout << endl;
            cout << endl;
            cout << "Nth node from beginning , where 0 is the first node(0 based
indexing):" << endl;
            int n;
            cout << "Enter the node: ";
            cin >> n;
            if(getNthNode2(&head,n)==-1){
                cout <<"Error:Invalid Index" << endl
            }
            else{
                cout << "The " << n << "th node is from the beginning: " <<
getNthNode2(&head, n) << endl;
            }
            cout << endl;
            cout << endl;
            break;
        }
```

Program Analysis:

There are two criteria of input Linked List:

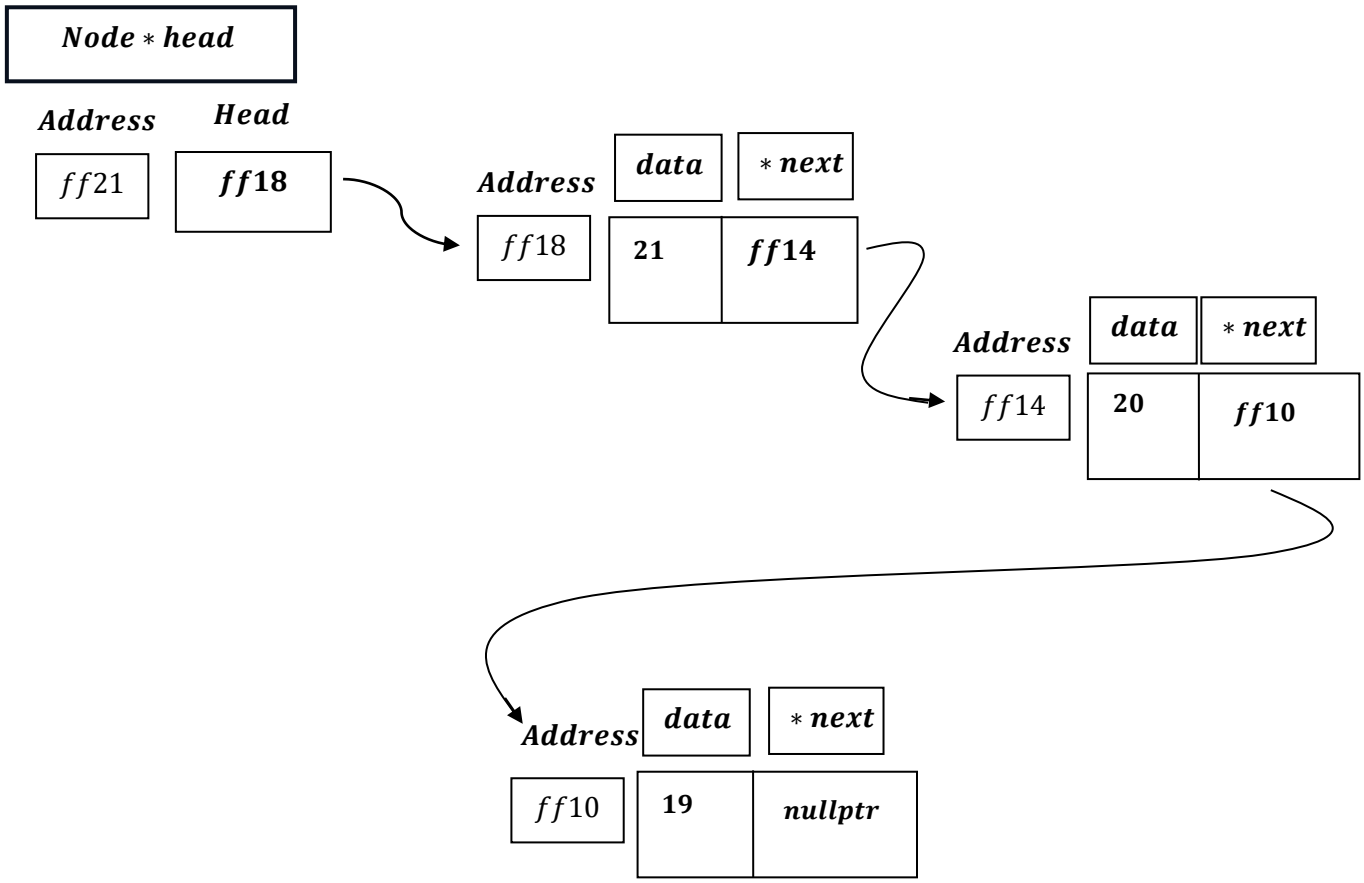
1) *Linked List is Empty*:

Pictorially depicted as:



2) *Linked List is not Empty*:

Pictorially depicted as:

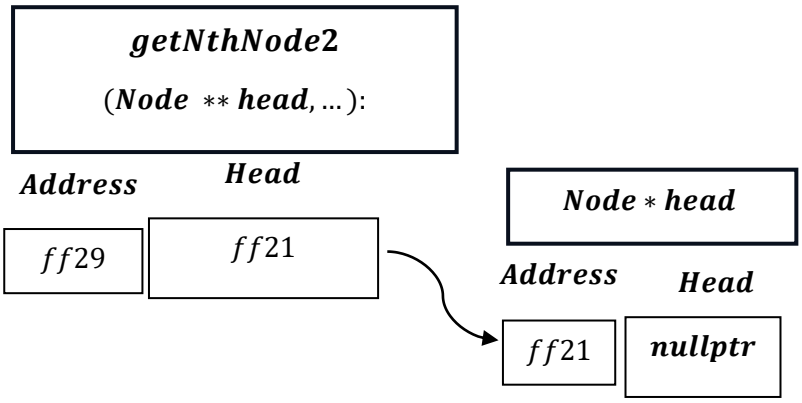


1) *When Linked List is Empty*:

```
if(getNthNode2(&head,n)==-1){
                cout <<"Error:Invalid Index" << endl
            }
```

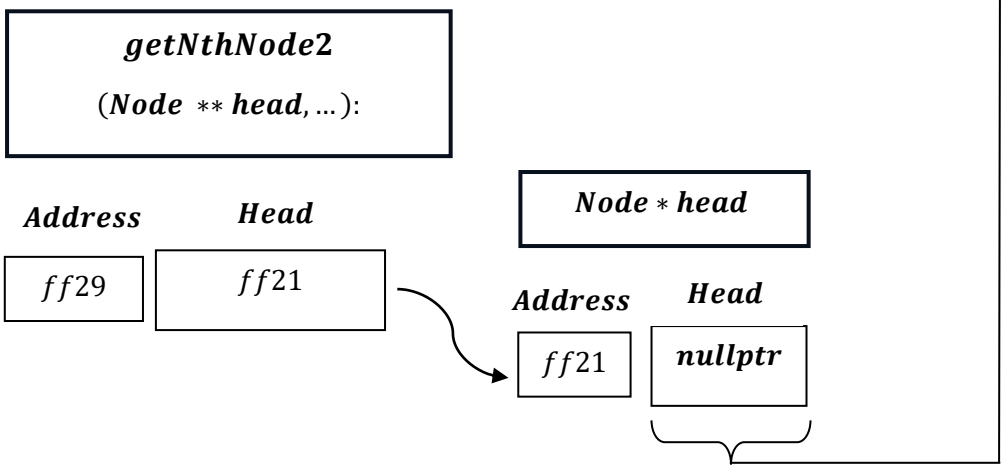
$getNthNode2(\&head{:}\,ff21, 0)$

```
int getNthNode2(Node **head, int node){….}
```



$if(*\,head == nullptr)$ *if it is true*

If true then Why true ?
*Ans: Though * head which is of Node ** head*
temporary pointer to pointer variable of the function
*, as * head is used , it dereference from Node ** head and*
*refer to Node * head global pointer to which Node ** head*
is pointing to and then its value is checked is
whether equal to nullptr or not ? This occurs during execution
and compilation of the getNthNode2 function.



```
if (*head == nullptr)
    {
        cout << "Error: List is empty." << endl;
        return -1;
    }
```

$If\ List\ is\ Empty\ , then\ \ , it\ prints\ a\ message:\ `Error: List\ is\ empty.`\ and\ returns - 1.$

$And\ as\ return - 1\ , hence\ it\ prints\ or\ outputs\ ``Error: Invalid\ Index.``\ .$

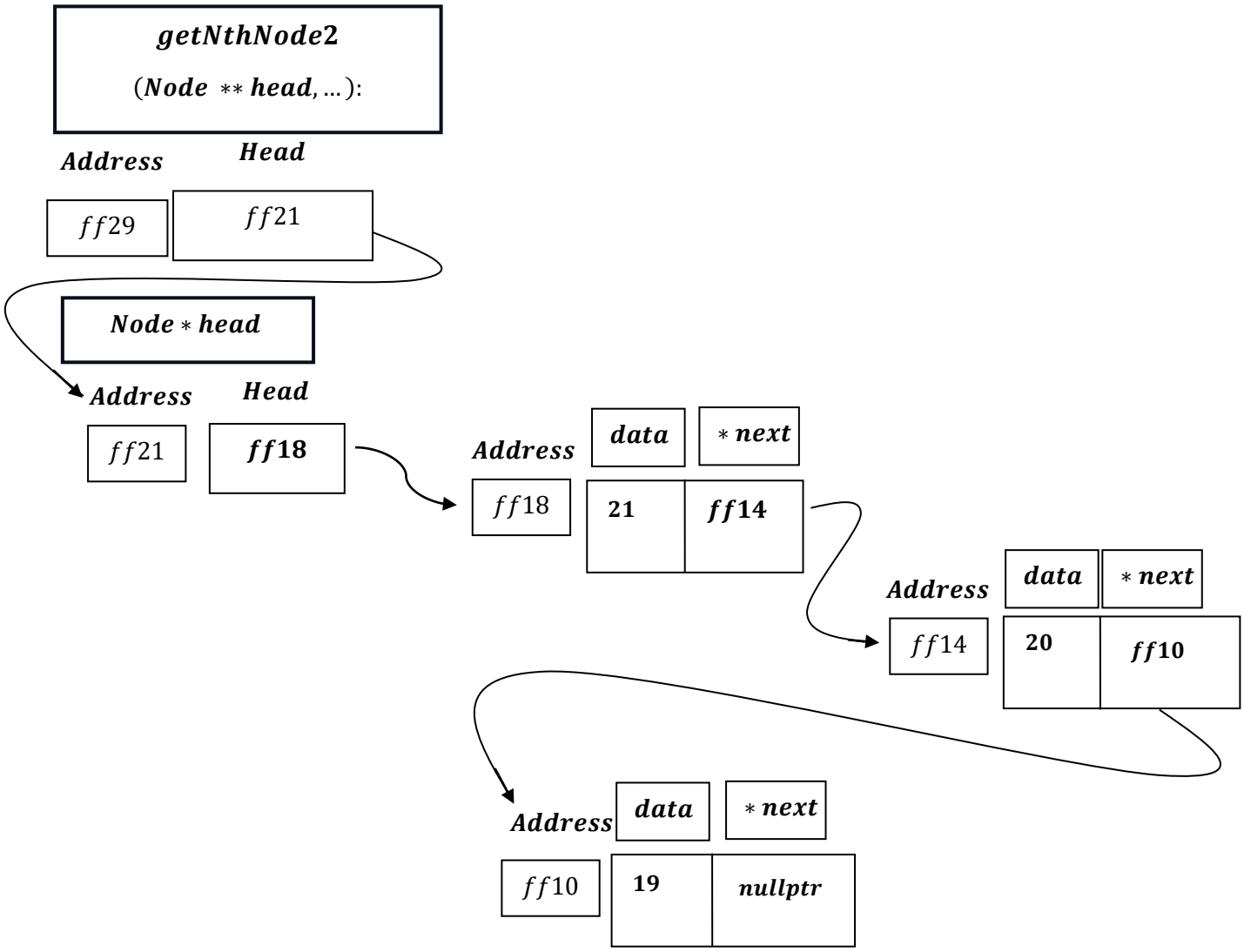
---

2)*When Node less than 0*

```
if(getNthNode2(&head,n)==-1){
        cout <<"Error:Invalid Index" << endl
}
```

$getNthNode2(\&head: ff21, -1);$

```
int getNthNode(Node **head, int node){….}
```



```
    if (node < 0)
    {
        return -1;
    }
```

$Hence\ , it\ returns - 1, And:$

```
if(getNthNode2(&head,n)==-1){
        cout <<"Error:Invalid Index" << endl
}
```

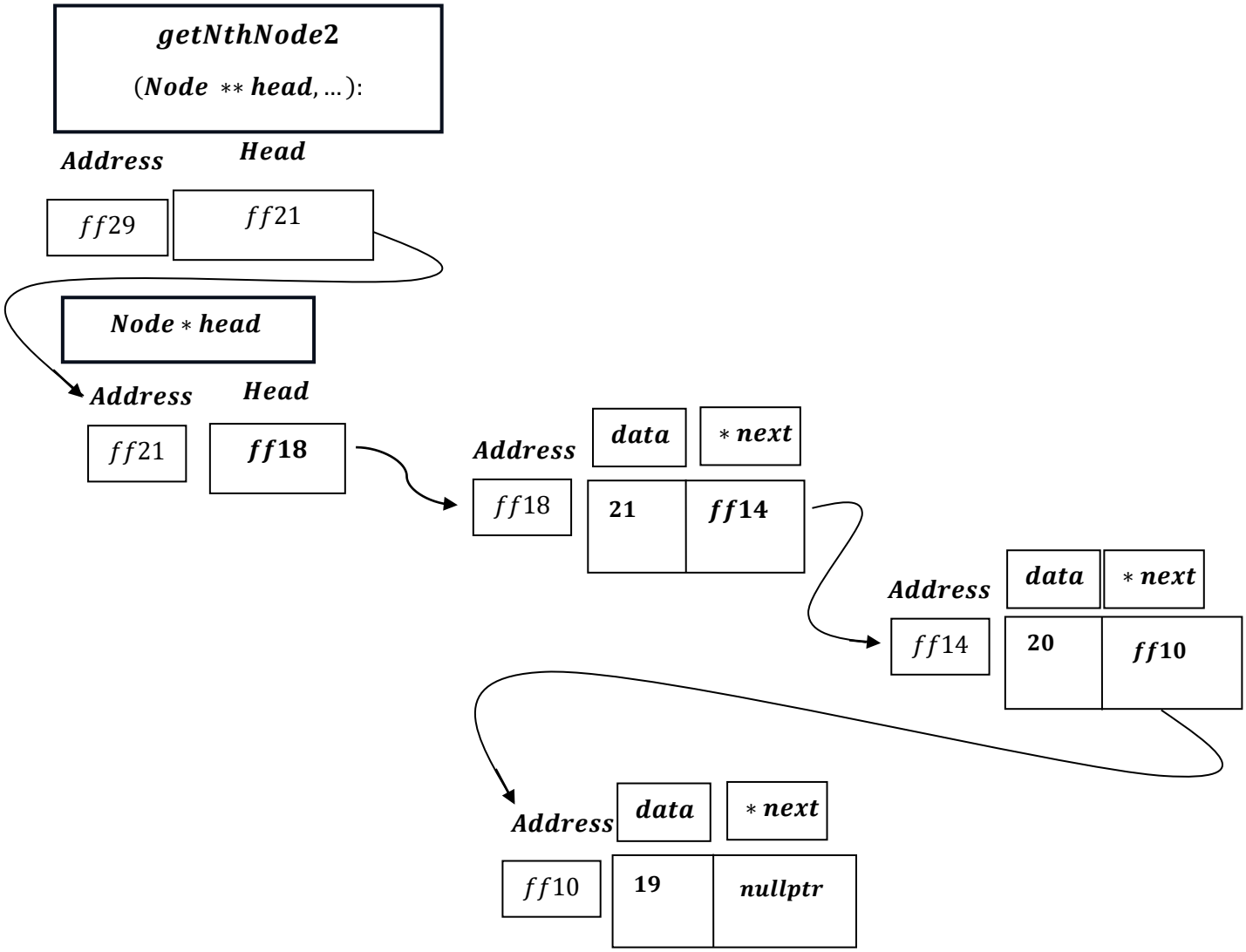
$Outputs: ``Error: Invalid\ Index.``.$

$3)\ When\ node\ is \geq 0\ and \leq length\ of\ List - 1$

```
if(……){….}else{
                cout << "The " << n << "th node is from the beginning: " <<
getNthNode2(&head, n) << endl;
            }
```

$getNthNode2(\&head: ff21, 2);$

```
int getNthNode2(Node **head, int node){….}
```



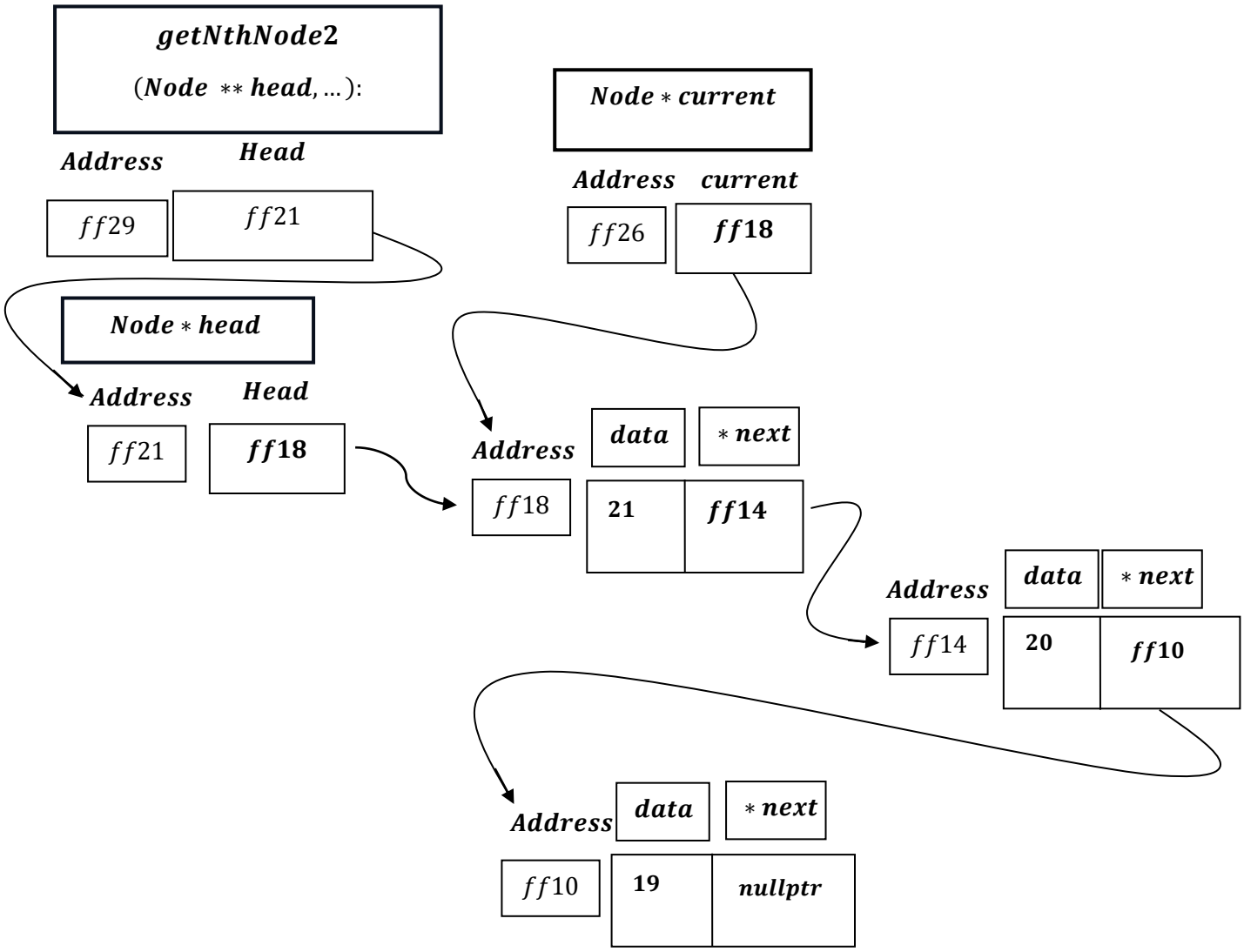

```
else
    {
        Node *current = *head;
        int i = 0;
        while (i < node && current != nullptr)
        {
            current = current->next;
            i++;
        }
```

```
        if(current == nullptr){
            return -1;
        }
        else{

            return current->data;
        }
    }
```

The current pointer assigned to value of head pointer .

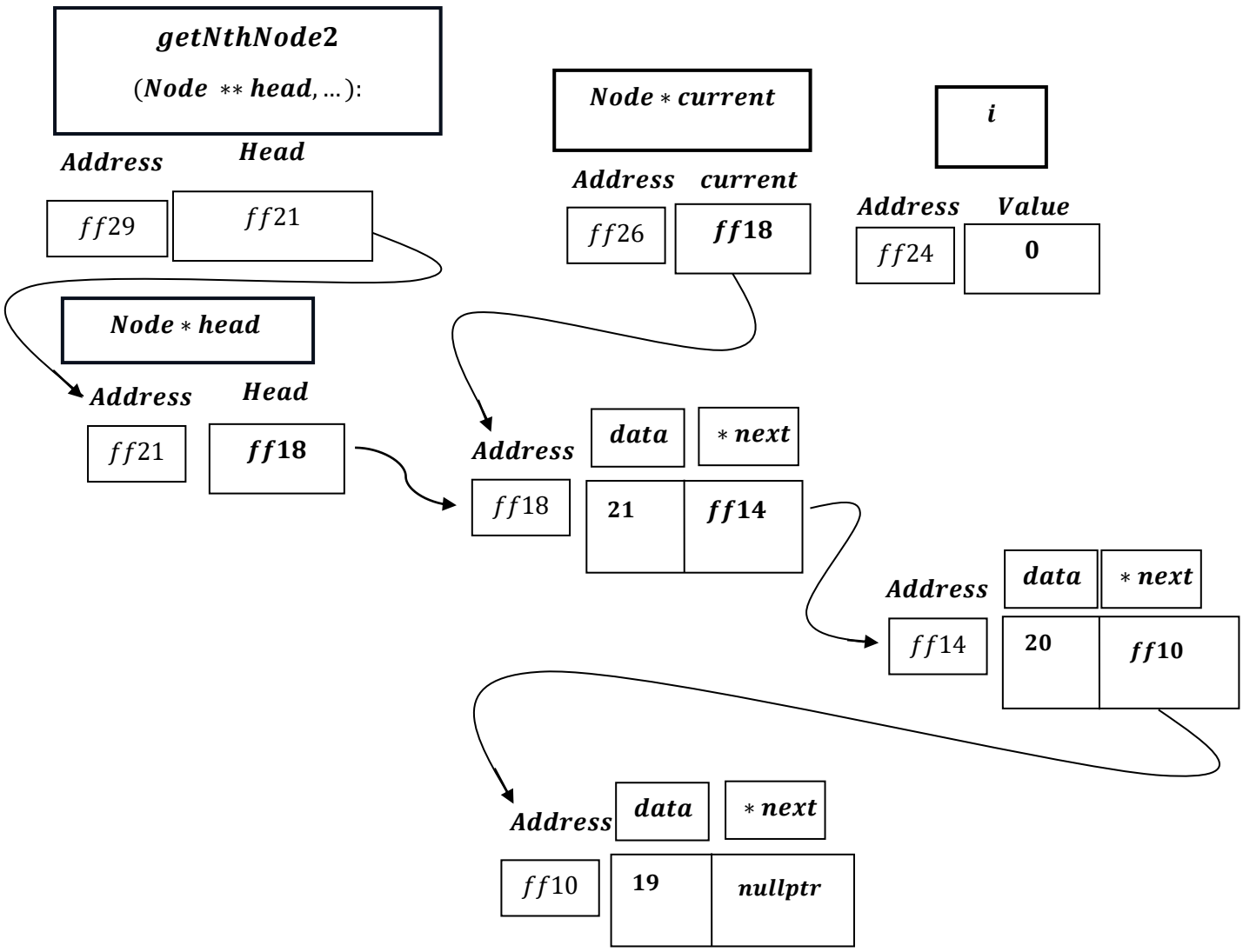
```
        Node *current = *head;
```

depicted below:



Now, i is assigned to 0;

```
        int i = 0;
```



Next,

```
while (i < node && current != nullptr)
        {
            current = current->next;
            i++;
        }
```

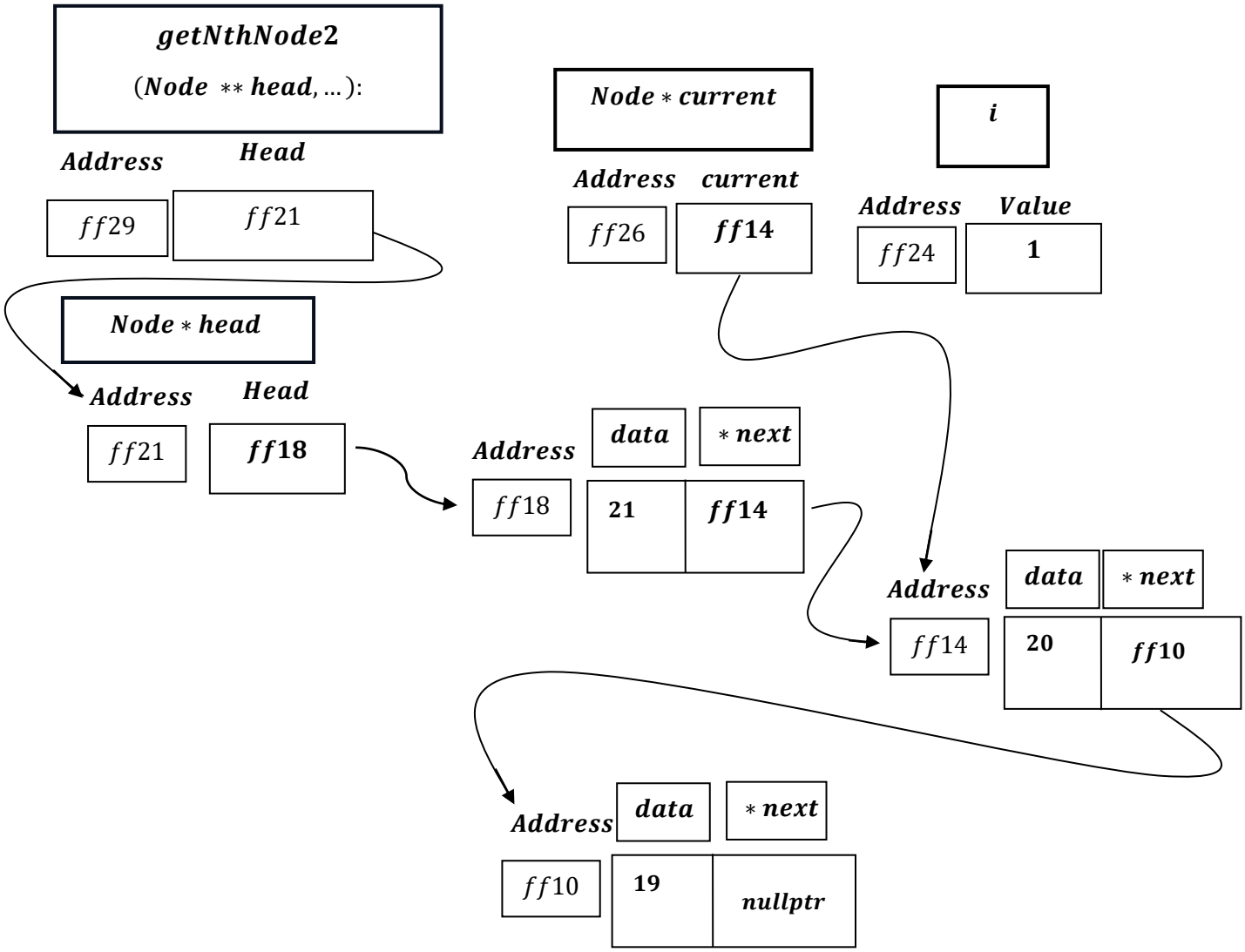
$Now, we\ notice\ that\ , current\ cannot\ be\ nullptr\ as\ node \geq 0\ and$

$node \leq length\ of\ List - 1, as\ shown\ below$:

$When\ i = 0 < node = 2[True], loop\ runs$:

$current = current \rightarrow next = ff14$.

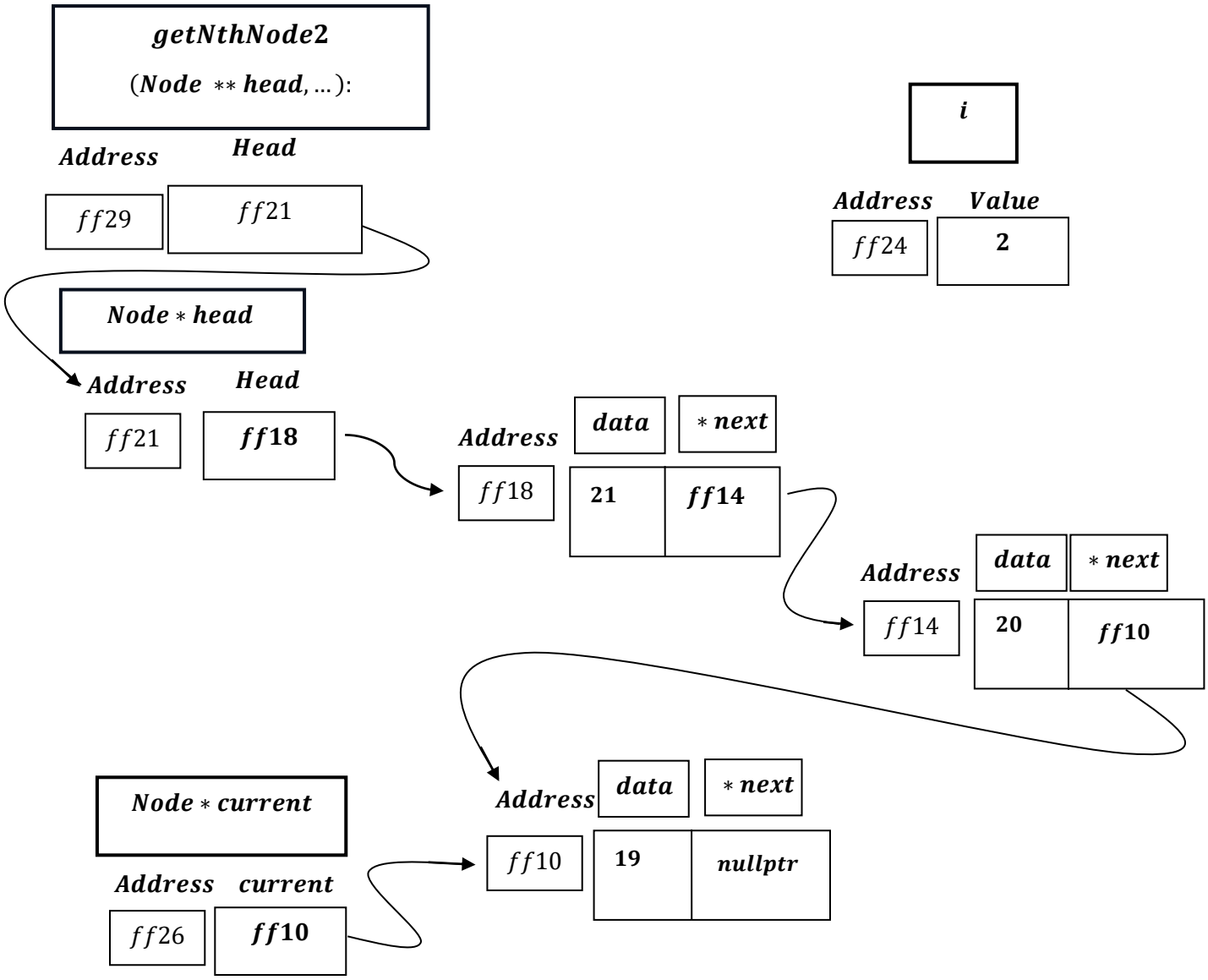
$i++ \Longrightarrow i = i + 1 = 1 + 1 = 1$.



$When\ i = 1 < node = 2[True], loop\ runs$:

$current = current \rightarrow next = ff10$.

$i++ \Longrightarrow i = i + 1 = 1 + 1 = 2$.



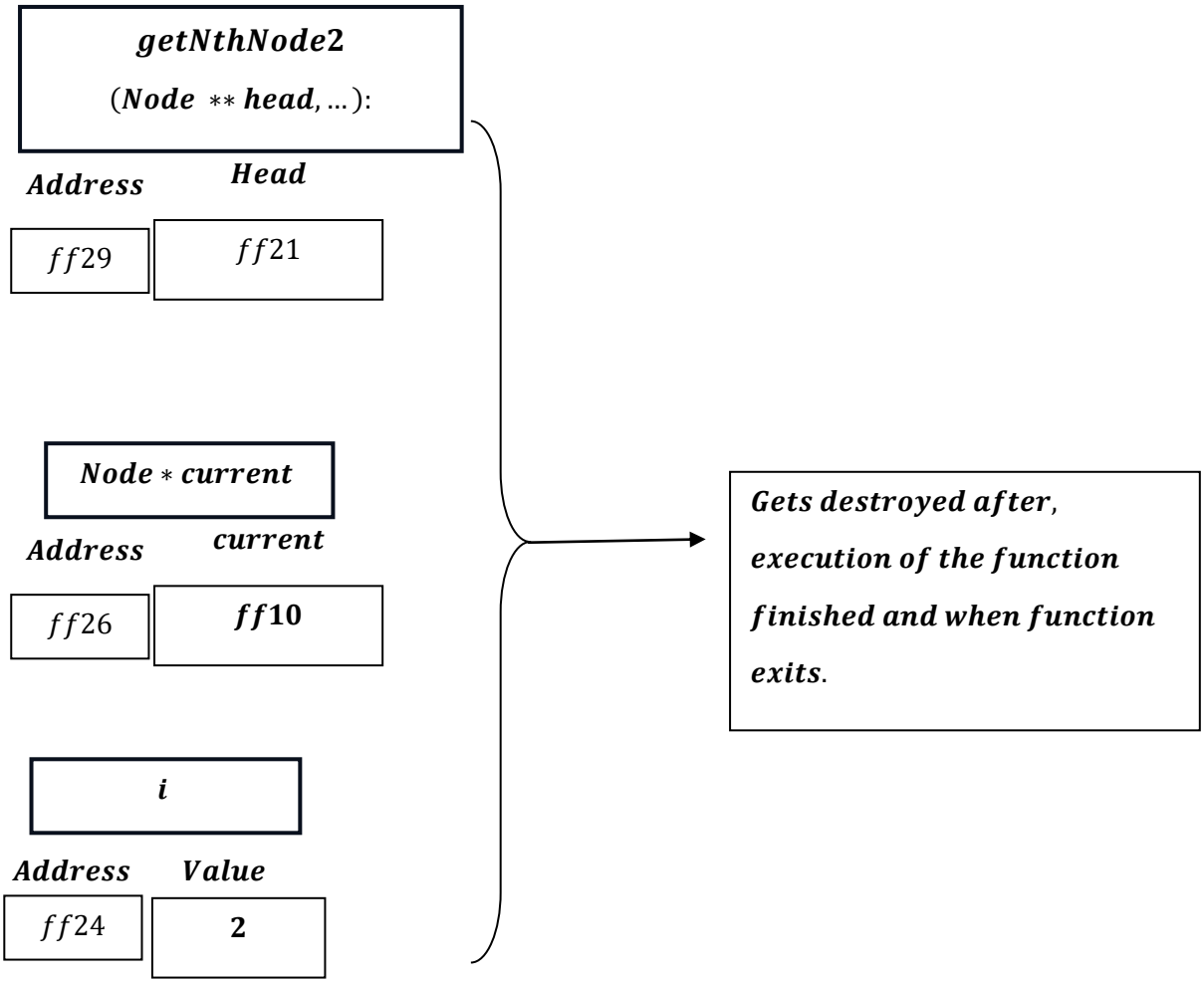
$$When\ i = 2 < node = 2[False], loop\ exits.$$

$$Hence\ , return\ current \rightarrow data\ i.e. 19.$$

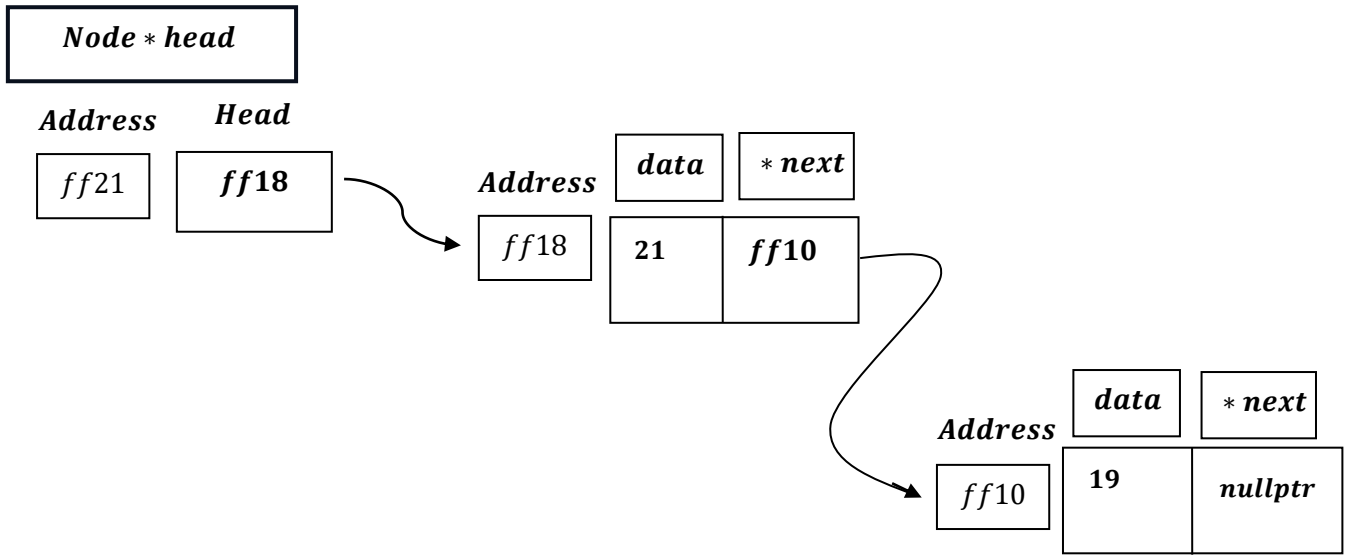
```
        return current->data;
```

After the function module ends , all temporary pointer variables created gets

$destroyed, i.e.$



Hence we are left with:



Now , coming to switch – case statement:

```
case 19:
        {
            cout << endl;
            cout << endl;
            cout << "Nth node from beginning , where 0 is the first node(0 based
indexing):" << endl;
            int n;
            cout << "Enter the node: ";
            cin >> n;
            if(getNthNode2(&head,n)==-1){

                cout <<"Error:Invalid Index" << endl;

            }
            else{
```

```
                cout << "The " << n << "th node is from the beginning: " <<
getNthNode2(&head, n) << endl;
            }
            cout << endl;
            cout << endl;
            break;
        }
```

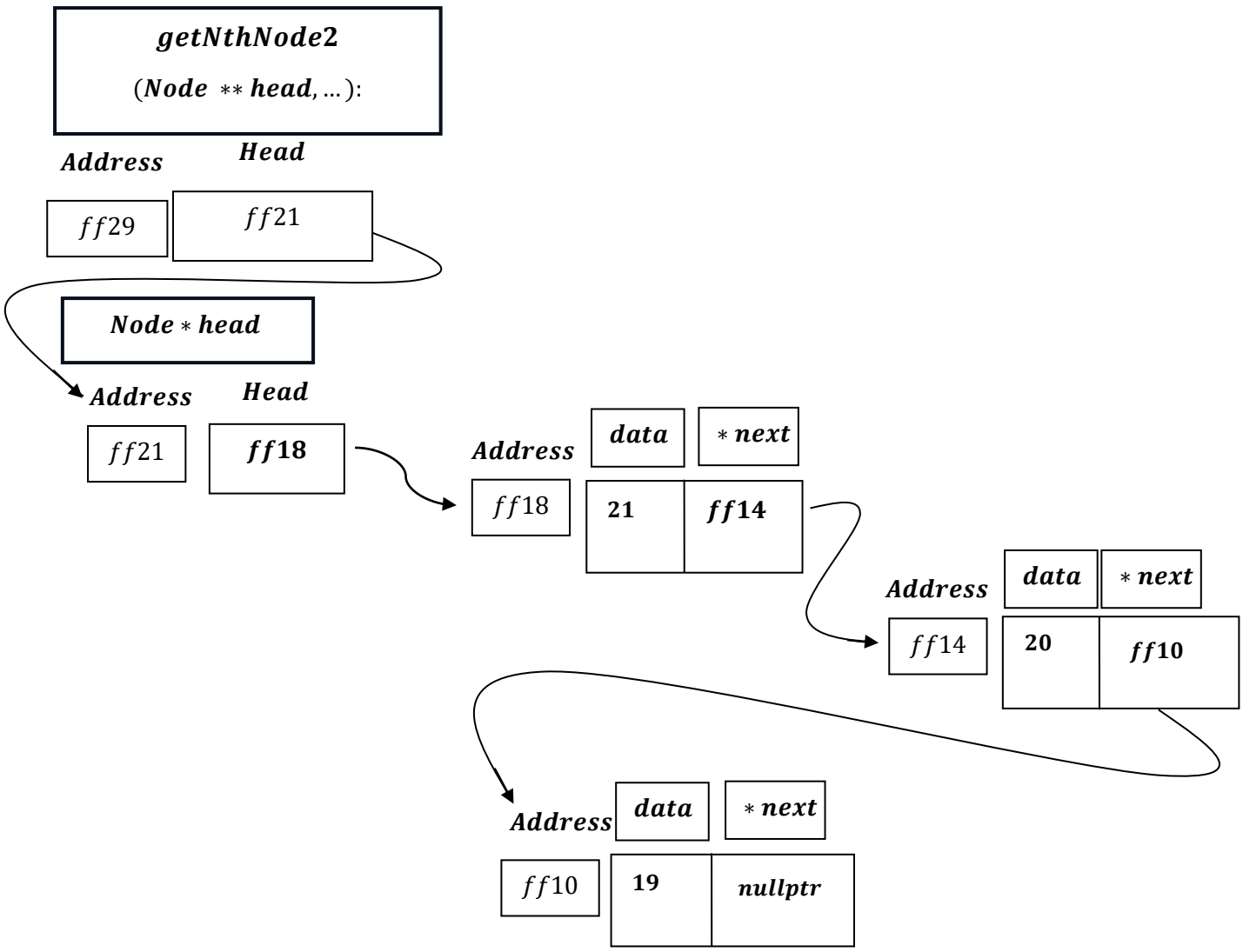
$It\ prints: The\ 2th\ node\ is\ from\ beginning: 19.$

$3)\ When\ node > length - 1:$

```
    else
    {
        Node *current = *head;
        int i = 0;
        while (i < node && current != nullptr)
        {
            current = current->next;
            i++;
        }
        if(current == nullptr){
            return -1;
        }

        else{
            return current->data;
        }
    }
}
```

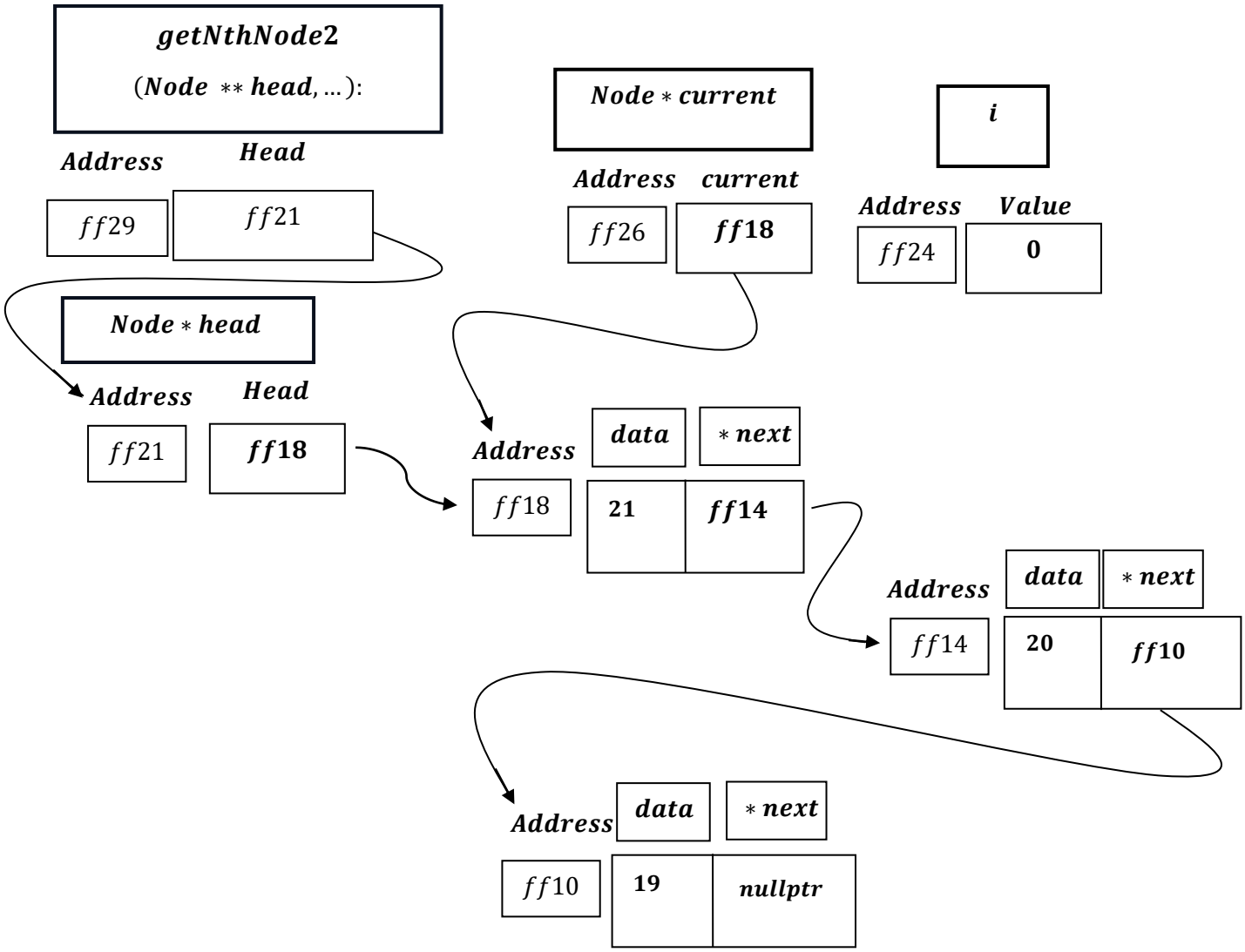
$getNthNode2(\&head: ff21, 3);$

```
int getNthNode2(Node **head, int node){….}
```



$Now, i\ is\ assigned\ to\ 0;$

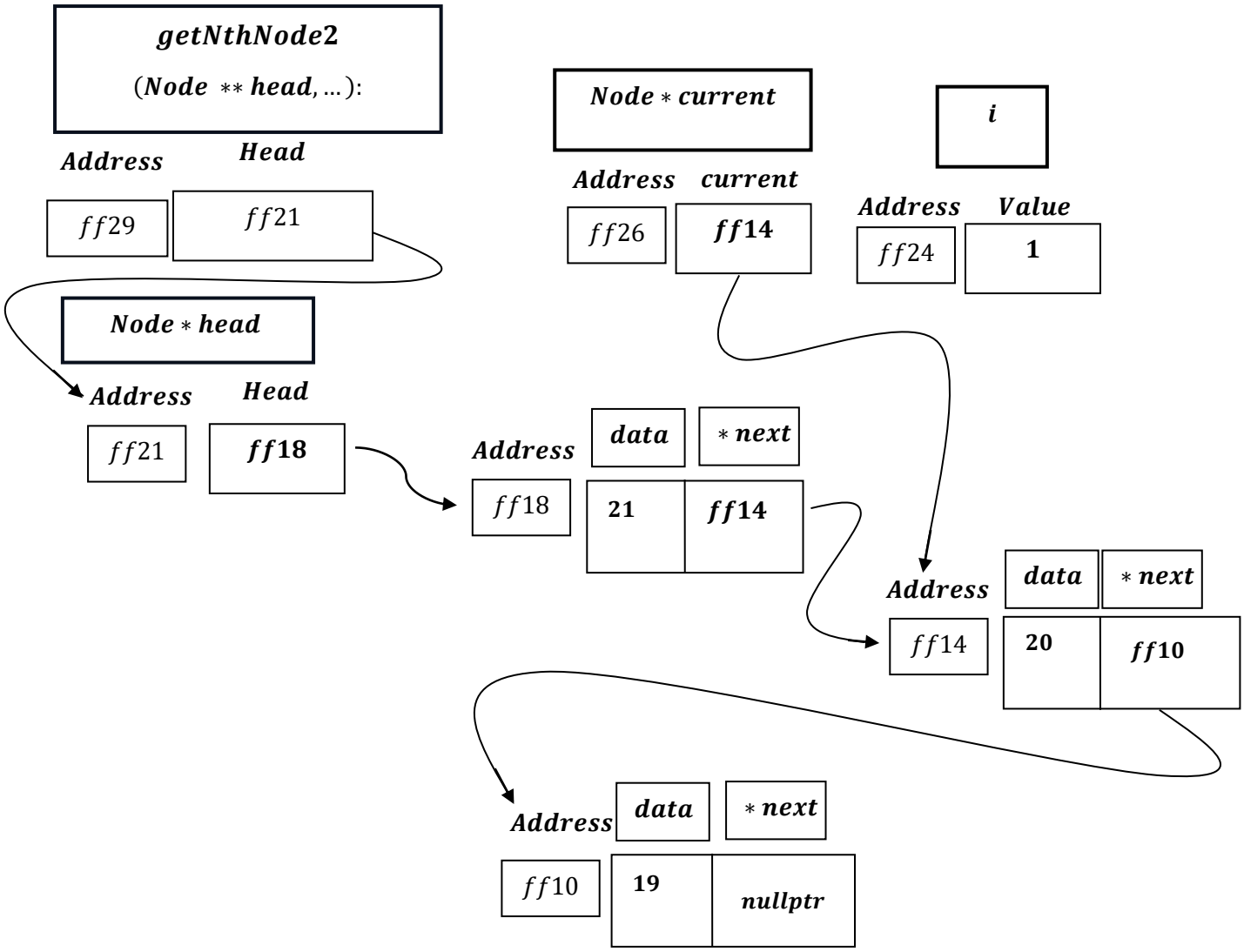
```
Node *current = *head;
int i = 0;
```



$When\ i = 0 < node = 3[True], loop\ runs$:

$current = current \rightarrow next = ff14$.

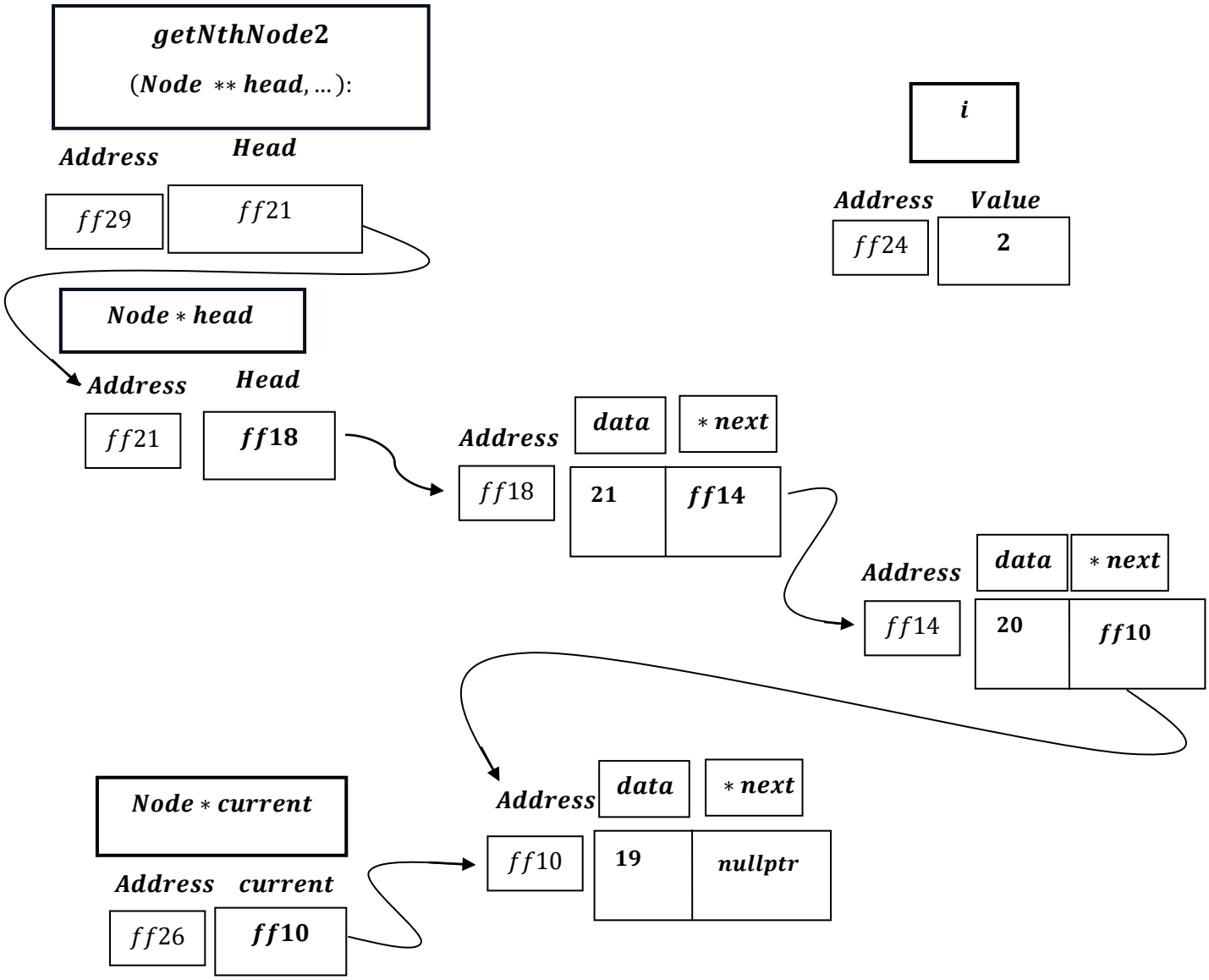
$i++ \Rightarrow i = i + 1 = 1 + 1 = 1$.



$When\ i = 1 < node = 3[True], loop\ runs$:

$current = current \rightarrow next = ff10$.

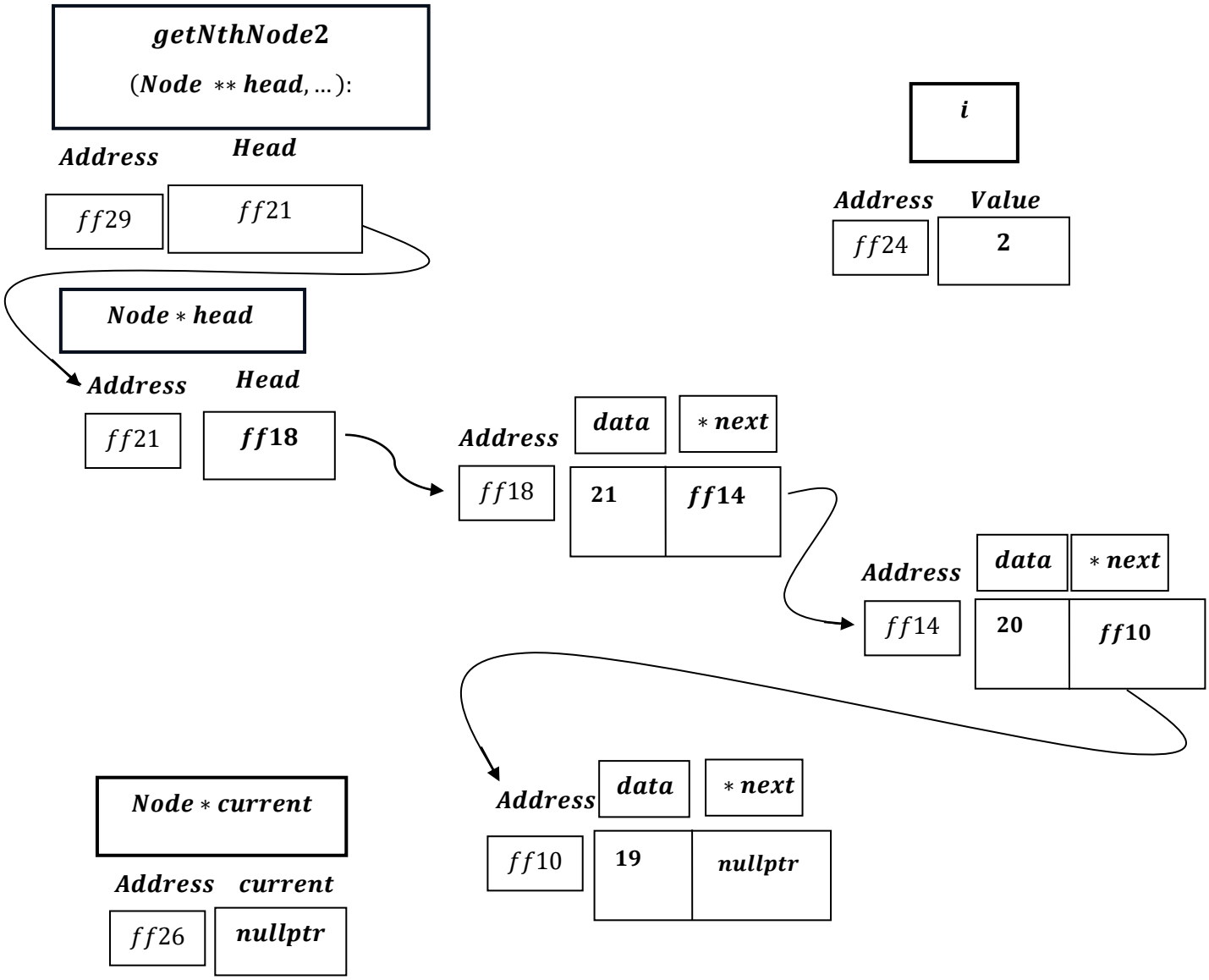
$i++ \Longrightarrow i = i + 1 = 1 + 1 = 2$.



$When\ i = 2 < node = 3[True], loop\ runs$:

$current = current \rightarrow next = nullptr$.

$i++ \Longrightarrow i = i + 1 = 2 + 1 = 3$.



Now as soon as current pointer assigned to ``nullptr`` , hence in loop:

current! = nullptr will become false and it will exit from the loop, also

i = 3 < node = 3 becomes false, for larger node , even if this doesnot

become false , current! = nullptr will become false , hence the condition,

current! = nullptr , condition is necessary.

Then in the function getNthNode2() returns ` − 1` due to the condition given below:

```
else
    {…}

        if(current == nullptr){
            return -1;
        }
        else{…}
    }
```

Now from switch − case we get output:

```
if(getNthNode2(&head,n)==-1){
                cout <<"Error:Invalid Index" << endl;
            }
```

``Error: Invalid Index`` as soon as getNthNode() function returns − 1.

Now, `getNthNode2` step by step explanation :

1. *Check if the List is Empty* :

− *The function first checks if the list is empty by verifying if the head pointer is nullptr.*

−*If the list is empty, it prints an error message and returns − 1..*

2. *Check for Invalid Node Index*:

− *The function checks if the provided node index is negative.*

−*If the list is empty, it prints an error message and returns − 1.*

3. *Initialize Traversal*:

– A pointer current is initialized to point to the head of the list.

–An integer i is initialized to 0 to keep track of the current node index during traversal.

4. *Traverse the List*:

– The function enters a loop to traverse the list until it reaches the desired node index or the end of the list.

–In each iteration, it moves the current pointer to the next node and increments i.

5. *Check if Node Exists*:

– After the loop, the function checks if current is nullptr.

–If current is nullptr, it means the list has fewer nodes than the specified index, so it returns -1.

6. *Return Node Data*:

– If current is not nullptr, it means the desired node was found.

–The function returns the data of the current node.

Time Complexity of

Get N^{TH} NODE 2

```
int getNthNode(Node **head, int node)
{
    if (*head == nullptr)
    {
        cout << "Error: List is empty." << endl;
        return -1;
    }
    if (node <0)
    {
        cout << "Error: Invalid index." << endl;
        return -1;
    }
    else
    {
        Node *current = *head;
        int i = 0;
        while (i < node && current != nullptr)
        {
            current = current->next;
            i++;
        }
        if(current == nullptr){
            return -1;
        }
        else{
            return current->data;
        }
    }
}
```

$O(1)$

$O(1)$

$O(1)$

$O(1)$

runs $n-1$ time at worst case.

$runs: 1+1+1+\cdots(n-1) = O(n).$

$runs: 1+1+1+\cdots(n-1) = O(n).$

$O(1)$

$O(1)$

$\underline{\textit{Best Case}}$

$\underline{1.\textit{When List is Empty}}$:

$\rightarrow$ *If the list is empty (`* head` is `nullptr`), print an error message and return from the function. This takes* $O(1)$ *time or* $\Omega(1)$ *time .*

$\underline{2.\textit{When Node is less than 1}}$

If node less than 1 *, it prints :* `` *Error: Invalid index.* `` *and then return* `$-1$`.
This takes $O(1)$ *time or* $\Omega(1)$ *time .*

$\underline{\textit{Worst Case}}$

$\underline{1.\textit{When `node is greater than length of List}}$

$-$ *Head is not nullptr , hence this check takes* $O(1)$ *time complexity.*

$-$ *`node is not less than 1` , hence this check also takes* $O(1)$ *time complexity.*

$-$ *current pointer is assigned to head pointer's value , this takes* $O(1)$ *time complexity.*

$-$ *variable `i` gets assigned to `0` takes* $O(1)$ *time complexity.*

$-$*As loop runs upto`* $n-1$*` times to enter last node and current becomes `nullptr` and loop exists.*

$-$*Now , if current value is nullptr which is true , hence returns* -1 *, which takes* $O(1)$ *time complexity. And function exits here.*

Hence worst time complexity is : $O(1)+O(1)+O(1)+O(n-1)+O(1)=O(n-1)=O(n)$.

2. When node entered is the last node.

– The function first checks if the linked list is empty by checking if the head pointer is `nullptr`. As this is not true at this time , the check takes $O(1)$ i.e. constant time.

– The function then checks if the provided index (`node`) is valid. As node is less than 0, hence this check also returns false, and takes constant time : $O(1)$.

– Function then initializes a pointer `current` to point to the head of the list. This takes constant time : $O(1)$.

– It also initializes a counter variable `i` to 0, representing the current position in the list. This takes constant time : $O(1)$.

– The `while` loop runs until `i` is less than the specified index (`node`), the loop runs `$n-1$` times :

– Inside the loop, it moves the `current` pointer to the next node (`current = current → next`). It takes : $1+1+1+\cdots(n-1)$ times i.e. $n-1$ times.

– It increments the counter `i` by 1 (`i++`). It takes : $1+1+1+\cdots(n-1)$ times i.e. $n-1$ times.

Hence, total time complexity: $n-1+n-1=2n-2=2\times O(n-1)=O(n)-O(1)=O(n)$.

– Finally, the function returns the data of the node at the specified index by accessing `current → data`. This takes constant time, i.e. $O(1)$.

Hence total time complexity: – $O(1)+O(1)+O(1)+O(1)+O(n)+O(1)=O(n)$.

Hence worst time complexity is : $O(n)$.

Average Case

$(Type\ 1\ Based\ on\ Loop)$

$\rightarrow If\ the\ position\ is\ 0, the\ loop's\ inner\ statement\ runs\ 0\ time.$

$\rightarrow If\ the\ position\ is\ 1, the\ loop's\ inner\ statement\ runs\ 1\ time.$

$\rightarrow If\ the\ position\ is\ 2, the\ loop's\ inner\ statement\ runs\ \ 2\ time.$

.

.

$\rightarrow If\ the\ position\ is\ (N-1), the\ loop's\ inner\ statement\ runs\ (N-1)\ times.$

$Hence\ , \sum_{i=1}^{N-1} i = \frac{N(N-1)}{2}.$

$Now, as\ we\ take\ all\ the\ possibilites\ then\ , we\ take\ the\ best\ case\ also\ i.e.$

$O(1), when\ position = \ 0\ . Considering\ `T(P)`\ is\ time\ complexity\ of$

$position\ `P`\ .$

$$T(P) = \begin{cases} 1 & , when\ P = 0. \\ P & , when\ P = 1, 2, 3, .., N-1. \end{cases}$$

$Now, If\ there\ is\ `N`\ no. of\ nodes\ there\ we\ can\ find\ data\ at\ \ N\ positions.$

$For\ position\ 1, the\ probability\ of\ finding\ data\ is: \frac{1}{N}.$

$For\ position\ 2, the\ probability\ of\ finding\ data\ \ is: \frac{1}{N}.$

.

.

$$\boldsymbol{For\ position\ N, the\ probability\ of\ finding\ data\ \ is}: \frac{\mathbf{1}}{\boldsymbol{N}}.$$

$$\boldsymbol{Hence\ probability\ of\ finding\ data\ \ will\ always\ remain}: \frac{\mathbf{1}}{\boldsymbol{N}}.$$

$$\therefore\ \boldsymbol{Average\ Time\ Complexity\ \ is}:\ \boldsymbol{probability\ of\ finding\ data} \times (\boldsymbol{number\ of\ possible\ iterations} + \boldsymbol{constant\ time\ complexities\ for\ left\ over\ positions})\ .$$

$$= \left(\frac{\mathbf{1}}{\boldsymbol{N}} \times \mathbf{1} + \frac{\mathbf{1}}{\boldsymbol{N}} \times \mathbf{2} + \cdots + \frac{\mathbf{1}}{\boldsymbol{N}} \times \boldsymbol{N} - \mathbf{1}\right) + \frac{\mathbf{1}}{\boldsymbol{N}} \times \mathbf{1}$$

$$= \frac{\mathbf{1}}{\boldsymbol{N}} \times \{(\mathbf{1} + \mathbf{2} + \mathbf{3} + \cdots + \boldsymbol{N} - \mathbf{1}) + \mathbf{1}\}$$

$$= \frac{\mathbf{1}}{\boldsymbol{N}} \times \left\{\sum_{\boldsymbol{i}=\mathbf{1}}^{\boldsymbol{N}-\mathbf{1}} \boldsymbol{i} + \mathbf{1}\right\}$$

$$= \frac{\mathbf{1}}{\boldsymbol{N}} \times \left(\frac{\boldsymbol{N}(\boldsymbol{N} - \mathbf{1})}{\mathbf{2}} + \mathbf{1}\right)$$

$$= \frac{\boldsymbol{N} - \mathbf{1}}{\mathbf{2}} + \frac{\mathbf{1}}{\boldsymbol{N}}$$

$$\boldsymbol{Appplying\ Big\ Theta\ in\ the\ equation}:$$

$$= \boldsymbol{\Theta}\left(\frac{\boldsymbol{N} - \mathbf{1}}{\mathbf{2}}\right) + \boldsymbol{\Theta}\left(\frac{\mathbf{1}}{\boldsymbol{N}}\right)$$

$$\boldsymbol{Now, taking\ out\ the\ dominant\ term\ of\ numerator\ and\ denominator}:$$

$$= \Theta\left(\frac{N}{2}\right) + \Theta\left(\frac{1}{N}\right)$$

$$= \frac{1}{2} \times \Theta(N) + \Theta\left(\frac{1}{N}\right)$$

$Now\ removing\ the\ constant\ `\frac{1}{2}`\ we\ get$:

$$= \Theta(N) + \Theta\left(\frac{1}{N}\right)$$

$When\ combining\ two\ Big\ Theta\ \Theta$ **terms , largest growth rate is the result**:

$\rightarrow \Theta(N)\ grows\ linearly\ with\ `N`$.

$\rightarrow \Theta\left(\frac{1}{N}\right)\ decreases\ to\ a\ constant\ , i.e.\ when\ N\ grows\ larger\ , \frac{1}{N}\ approaches\ to\ 0.$

$This\ we\ can\ prove\ by\ limit\ and\ continuity\ also$:

$$\lim_{n\to\infty}\left(\frac{1}{n}\right) \Rightarrow \frac{1}{\infty} = 0\ \left[\frac{c}{\infty} = 0\ , where\ c\ is\ constant\right].$$

$As\ \Theta(N)'s\ growth\ rate\ is\ higher\ , therefore\ , average\ case\ is\ \Theta(N).$

$Hence, Average\ Time\ Complexity\ is:\ \Theta(N).$

$Note\ , by\ relation: \Omega \leq \Theta \leq O\ , we\ get$:

$$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq \frac{N-1}{2} + \frac{1}{N} \leq c_2 \times N$$

$$\boldsymbol{Average\ Case\ Time\ Complexity[Most\ Simplest\ Alternative\ Apporach]:}$$

$$\boldsymbol{(Simplest\ approach)}$$

$\boldsymbol{For\ position\ 1, the\ probability\ of\ finding\ data\ is: \frac{1}{N}}$.

$\boldsymbol{For\ position\ 2, the\ probability\ of\ finding\ data\ is: \frac{1}{N}}$.

.

.

$\boldsymbol{For\ position\ N, the\ probability\ of\ finding\ data\ is: \frac{1}{N}}$.

$\boldsymbol{Hence\ probability\ of\ finding\ data\ always\ remain\ \frac{1}{N}, for\ position\ 1\ to\ N}$.

$$\boldsymbol{Average\ case\ complexity:\ Probability\ of\ finding\ data\ \times \begin{pmatrix} Summation \\ of\ number\ of \\ Position. \end{pmatrix}}$$

$$= \frac{1}{N} \times \left(\sum_{i=1}^{N} i \right)$$

$$= \frac{1}{N} \times \left(\frac{N(N+1)}{2} \right)$$

$$= \frac{N+1}{2}$$

$$or, \frac{N}{2} + \frac{1}{2}$$

$\boldsymbol{Appplying\ Big\ Theta\ in\ the\ equation}$:

$$= \Theta\left(\frac{N}{2}\right) + \Theta\left(\frac{1}{2}\right)$$

$\boldsymbol{Now\ removing\ the\ constant\ `\frac{1}{2}`\ we\ get}$:

$$= \Theta(N) + \Theta(1)$$

$$= \Theta(N).$$

$$\boldsymbol{Hence, Average\ Time\ Complexity\ is:}\ \Theta\left(\frac{N+1}{2}\right)\ \boldsymbol{or}\ \Theta(N).$$

$\boldsymbol{Now,}$

$\boldsymbol{Note\ , by\ relation}: \Omega \leq \Theta \leq O\ , \boldsymbol{we\ get}$:

$$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq \frac{N+1}{2} \leq c_2 \times N$$

Singly Linked List	***Cases***	***Time Complexity***
Get Nth Node 2	**1.** ***Best Case***	$\Omega(1)$
	2. ***Worst Case***	$O(n)$
	3. ***Average Case***	$\Theta(n)$

****